

Real-Time Decoding of Snapshot Compressive Imaging Using Tensor FISTA-Net

Xiao-Yang Liu¹, *Member, IEEE*, Qifan Huang, Xiaochen Han, Bo Wu², Linghe Kong³, *Senior Member, IEEE*, Anwar Walid⁴, *Fellow, IEEE*, and Xiaodong Wang⁵, *Fellow, IEEE*

Abstract—Snapshot compressive imaging (SCI) cameras compress high-speed videos or hyperspectral images into measurement frames. However, decoding the data frames from measurement frames is compute-intensive. Existing state-of-the-art decoding algorithms suffer from low decoding quality or heavy running time or both, which are not practical for real-time applications. In this article, we exploit the powerful learning ability of deep neural networks (DNN) and propose a novel tensor fast iterative shrinkage-thresholding algorithm net (Tensor FISTA-Net) as a real-time decoder for SCI cameras. Since SCI cameras have an accurate physical model, we can trade training time for the decoding time by generating abundant synthetic data and training a decoder on the cloud. Tensor FISTA-Net not only learns a sparse representation of the frames through convolution layers but also reduces the decoding time and memory consumption significantly through tensor operations, which makes Tensor FISTA-Net an appropriate approach for a real-time decoder. Our proposed Tensor FISTA-Net obtains an average PSNR improvement of 0.79–2.84 dB (video images) and 2.61–4.43 dB (hyperspectral images) over the state-of-the-art algorithms, along with more clear and detailed visual results on real SCI datasets, *Hammer* and *Wheel*, respectively. Our Tensor FISTA-Net reaches 45 frames per second in video datasets and 70 frames per second in hyperspectral datasets, meeting the real-time requirement. Besides, the trained model occupies only a 12-MB memory footprint, making it applicable to real-time Internet of Things (IoT) applications.

Index Terms—Deep unfolding, real-time decoding, snapshot compressive imaging (SCI), tensor FISTA-net, training at workstation.

I. INTRODUCTION

INSPIRED by the rapidly growing compressive sensing technique [1], [2], [3], snapshot compressive imaging (SCI) camera and pixelwise coded exposure camera [4], [5]

are two promising cameras designed for efficient imaging. SCI [6], [7], [8] has been viewed as an important approach to capture multiframe images in many real-life cases such as hyperspectral images, car crashing experiments, sports events, and high-accuracy physical experiments. Since SCI cameras use off-the-shelf sensors with fixed physical models to compress multiple frames into a single measurement frame, we can take a deep learning approach to develop a real-time decoder for an integrated SCI camera. For the development of real-time decoders, decoding quality and running time need to be simultaneously considered, which is challenging.

The existing algorithms for SCI decoding meet some of the requirements of a real-time decoder. However, they are still not satisfactory enough due to either heavy running time or noisy decoded images. Gaussian mixture model (GMM) [9], [10] was the first one applied to decode the compressed images, but it suffered from blurry images along with low decoding quality because it only exploited the sparsity of the high-dimensional image frames without taking other features into consideration. DeSCI [11] used a rank minimization process to achieve state-of-the-art decoding quality. However, since it used rank minimization and nonlocal patch searching, it suffered from very heavy running time (103 min for decoding a single measurement). Therefore, DeSCI is not suitable for real-time applications. It is of great significance to develop a novel approach to better balance the running time and decoding quality and meet the requirements of real-time applications.

The fast iterative shrinkage threshold algorithm [12], FISTA, can potentially be applied to real-time SCI decoding because it can speed up the convergence of the iteration process significantly. However, directly applying the FISTA algorithm may involve exhaustive parameter tuning and time-consuming large-scale matrix operations. Considering the excellent learning capability of deep neural networks (DNNs) and exploiting the reduction in calculations in tensor form, we propose a novel approach called Tensor FISTA-Net for the SCI decoding problem. We unfold the FISTA algorithm into a DNN inspired by [13], [14], [15], [16] and use tensor-form calculations instead of matrix-form calculations.

Our framework is originally motivated by the concept of edge servers in IoT systems, which can serve as a local training server for nearby mobile devices. However, since our training task does not require much memory space, a moderate workstation is sufficient for performing the training task independently. Fig. 1 illustrates an overview of the proposed training method which uses Tensor FISTA-Net as the decoder

Manuscript received 11 May 2021; revised 12 May 2022, 26 September 2022, and 4 March 2023; accepted 27 March 2023. Date of publication 2 June 2023; date of current version 8 October 2024. (Corresponding author: Xiaodong Wang.)

Xiao-Yang Liu and Xiaodong Wang are with the Department of Electrical Engineering, Columbia University, New York, NY 10027 USA (e-mail: xl2427@columbia.edu; xw2008@columbia.edu).

Qifan Huang is with the State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Science, Beijing 100001, China, and also with the Institute of Software, University of Chinese Academy of Science, Beijing 100001, China (e-mail: huangqifan21@mails.ucas.ac.cn).

Xiaochen Han and Linghe Kong are with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: guillermohan97@sjtu.edu.cn; linghe.kong@sjtu.edu.cn).

Bo Wu is with the MIT-IBM Watson AI Laboratory, Cambridge, MA 02142 USA (e-mail: bo.wu@ibm.com).

Anwar Walid was with Nokia Bell Labs, New Providence, NJ 07974 USA. He is now with Point72, New York, NY 10001 USA, and also with the Department of Electrical Engineering, Columbia University, New York, NY 10027 USA.

Digital Object Identifier 10.1109/TNNLS.2023.3266998

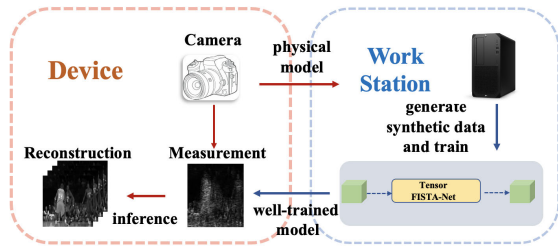


Fig. 1. Overview of Tensor FISTA-Net. The physical model of an SCI camera is stored in the workstation. The workstation first uses the physical model to generate synthetic data and train a neural network. Then, the workstation sends the well-trained model to a device that decodes the measurement frames. Since the model of our Tensor FISTA-Net is small (e.g., 12 MB), bandwidth limitation and communication delay should not be an issue.

for SCI cameras. The physical model of an SCI camera, for example, the mask in CCD is first uploaded to a workstation. The workstation generates synthetic data to train the decoder (a neural network). Finally, the trained decoder is downloaded onto a mobile device.

The training process is performed at a workstation, and the decoding process involves only a forward pass; thus, a real-time decoder can be achieved. Since SCI cameras have explicit physical models, the cost of generating synthetic data in the workstation is low. We can produce a large amount of training data and train the model in a workstation. Besides, an SCI camera allows the encoder and the decoder to be separately stored so we can decode the measurements on demand. Moreover, the model size of a well-trained Tensor FISTA-Net is small (about 12 MB), so it is applicable for real-time IoT application. Therefore, our proposed Tensor FISTA-Net is a suitable real-time decoder for SCI cameras.

The main contributions are summarized as follows.

- 1) Since SCI cameras have accurate physical models that can be used to generate synthetic training data, we can trade training time for inference time. Therefore, we propose a novel Tensor FISTA-Net by unfolding the FISTA algorithm into a DNN, and the trained network model serves as a real-time decoder for SCI cameras.
- 2) We convert traditional matrix computations in the FISTA algorithm into tensor computations, which significantly reduces the decoding time and memory consumption so that it fits real-time scenarios better.
- 3) We carry out evaluation experiments on both video and hyperspectral images, including synthetic and real ones. The results show that Tensor FISTA-Net outperforms most of the benchmark algorithms in terms of reconstruction quality and running time. Besides, its small memory footprint (about 12 MB) indicates that it is promising for IoT devices with limited memory. Finally, on a video dataset, it takes 0.18 s to decode a single measurement frame, while on a spectral dataset, it takes only 0.3 s to decode one consisting of 24 single-channel images. Therefore, it can be concluded that our Tensor FISTA-Net can serve as a real-time decoder for SCI cameras.

Compared with the previous conference version [17], our changes in this journal version can be summarized as follows:

- 1) adding new experiments on hyperspectral SCI camera

data and 2) provide acceleration skills to achieve a real-time decoder. These changes ensure that this version has enough innovations and as small an overlap as possible.

The remainder of this article is organized as follows. In Section II, we review related works in designing decoders for SCI cameras. Section III introduces the structure and decoding problem of SCI cameras. Section IV proposes the Tensor FISTA-Net. In Sections V and VI, we present the results on synthetic and real datasets by video and hyperspectral SCI cameras. We conclude this work in Section VII.

II. RELATED WORKS

The decoding problem of SCI cameras is ill-posed. Various approaches with different priors have been proposed. The related works can be divided into two main categories, i.e., optimization and neural network methods.

A. Optimization Methods

Several optimization algorithms have been proposed for the SCI decoding problem. They mainly focus on exploiting the latent linear and nonlinear features of the high-dimensional image frames. However, a critical drawback of those algorithms is that they require too much time to decode a single measurement frame. Experiment data from [11] show that on average, it takes 103 min to decode a single frame. The algorithm in [18] requires 8 s to decode a single frame, which fails to meet the requirement of real-time decoding. Therefore, a server-based scheme in Fig. 1 cannot achieve real-time coding if the built-in decoding algorithm is one of those optimization-based algorithms.

Sparsity approaches were first applied to the decoding problem. GMMs were used in [9] and [10] to exploit the sparsity of the image patches. However, they suffered from low reconstruction quality since they failed to exploit the structure information hidden in the high-dimensional data.

DeSCI [11] used a rank-minimization process to exploit the structure information hidden in the collected data fully. DeSCI [11] achieved the best reconstruction quality in some datasets like Kobe. However, due to a nonlocal patch searching operation, DeSCI requires a tremendously long running time that makes it unsatisfactory. Furthermore, a tensor nuclear-norm minimization scheme was proposed in [19].

Shearlet-enhanced SCI (SeSCI) [20] is another model-based optimization algorithm. SeSCI used the shearlet transformation to obtain a sparse representation in both the frequency domain and the shearlet domain. However, SeSCI still suffers from blurry images in some detailed areas and low reconstruction quality at certain wavelengths of the hyperspectral images.

B. Deep Learning Methods

Deep learning approaches, including [15], [21], [22], [23], are recently developed to address the SCI decoding problem. Deep learning methods focus on exploiting the great learning capability of neural networks or unfolding traditional linear inverse algorithms into DNNs. Since deep learning approaches separate the training and testing stages, the decoder could be

pretrained in the workstation. Therefore, it takes a forward pass to decode the measurement frames and has the potential to achieve real-time decoding.

Transform-based tensor neural networks and tensor ISTA algorithm were used in [16] to synthesize videos with missing frames, which provides us inspiration for designing Tensor FISTA-Net. Deep fully connected layers were applied in [22] to learn the linear mapping between video sequences and corresponding measurement frames. However, those fully connected layers only learned the linear features of the transformation so it failed to extract more nonlinear features in the transformation. AMP-Net [21] used AMP algorithm because of its noise reduction features. AMP-Net also introduced a sampling matrix training method to improve the decoding quality. Tensor ADMM-Net [15] is one of the popular algorithms for real-time SCI imaging since it strikes a better balance between reconstruction quality and running time than other neural network approaches. However, it still suffers from blurry images because it only learns the linear transformation of video frames through fully connected layers. Meanwhile, Tensor FISTA-Net applies convolution and activation layers to learn more information in the nonlinear transformation of the data. In addition, Tensor ADMM-Net [15] still needs a lot of vector and matrix-form calculations during training and testing, which can be replaced by tensor form calculations in our Tensor FISTA-Net.

Decoding algorithms which apply plug-and-play (PnP) methods have become popular in recent years because various kinds of denoisers can be directly plugged into the primitive decoding algorithms. Yuan et al. [24] integrate various kinds of denoisers such as VBM4D, WNNM, and FFDNet together with originally applied algorithms to achieve relatively high accuracy and temporarily address the “trilemma” between accuracy, time, and memory. However, most PnP-based algorithms still lack the aid of DNNs, which results in longer decoding time than our *Tensor FISTA Net*. *PnP-FFDNet*, which uses DNN-based denoiser *FFDNet*, is the most suitable algorithm for real-time decoding of SCI images. However, the performance of our *Tensor FISTA Net* still slightly surpasses that of *PnP-FFDNet* in terms of reconstruction accuracy and running time, which ensures that our *Tensor FISTA Net* is the state-of-the-art algorithm for real-time decoding of SCI images. Qiu et al. [25] used the plug-and-play method to combine both TV and FFDNet together as a new denoiser and went closer to the optimal posterior. In addition to those denoising techniques, Zha et al. [26] proposed a hybrid structural sparsification error model to enhance restoration performance. Moreover, Zha et al. [27] have developed a novel algorithm for image restoration by combining group sparse representation and low-rank approximation. These algorithms also have the potential to be used as denoisers for PnP-based algorithms.

During the review period of this work, new algorithms based on recurrent neural networks have been proposed for SCI decoding problems, among which bidirectional recurrent neural network with adversarial training (BIRNAT) is a typical representative. BIRNAT [28], [29] applies a bidirectional recurrent neural network (RNN) with adversarial training

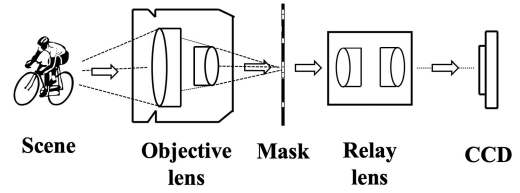


Fig. 2. Illustration of a video SCI camera. The spatial-variant mask shifts automatically to compress the original video frames into measurements and CCD collects the measurement frames. The mask and CCD are in the conjugate image plane of the scene.

methods to exploit the temporal correlations in decoded frames from the same measurement and further improve the performance of the network. A convolutional network-enhanced self-attention is used to generate a reference for subsequent RNN. Applying RNN in SCI decoding problems helps exploit the temporal and spatial correlations in adjacent frames. With the aid of RNN, BIRNAT achieves state-of-the-art decoding accuracy on the testing datasets. Although those algorithms based on RNNs achieve state-of-the-art reconstruction accuracy results and is also practical for real-time decoding, the complicated network structure lengthens the training time of the model. Therefore, our Tensor FISTA-Net is more user-friendly in real-time decoding.

III. DECODING PROBLEM IN SCI CAMERAS

In this section, we will first give an overview of two typical SCI cameras. Then the mathematical model of SCI cameras will be presented. In the last part, we will formulate the decoding problem of SCI cameras, which is a critical problem in the design of SCI cameras.

A. Overview of SCI Cameras

There are two kinds of typical SCI cameras, coded aperture compressive temporal imaging (CACTI) systems and coded aperture spectral snapshot imaging (CASSI) systems, respectively.

Video SCI cameras use off-the-shelf sensors to capture high-speed videos in [6], [30], and [31]. In video SCI cameras illustrated in Fig. 2, the high-speed video data frames are collected by the objective lens, and then spatially coded by a temporal variant mask, which is the critical device in the video SCI cameras. The mask automatically shifts and compresses multiple video frames into a single frame. In the last part, a charge-coupled device (CCD) [32], [33] collects the measurement frames.

Hyperspectral SCI cameras [8] are designed to capture hyperspectral images in real life. Fig. 3 shows the structure of a hyperspectral SCI camera. In hyperspectral SCI cameras, the colored images are first collected by an objective lens, and then spatially coded by a simple mask. After that, it goes through a dispersive element, usually an optical grating, to disperse the single-channel image into a multichannel one, producing the hyperspectral data, which are further collected by a CCD [32], [33].

B. Mathematical Model of SCI Cameras

We discuss mathematical models of SCI cameras. It is worth noting that video and hyperspectral SCI cameras share the

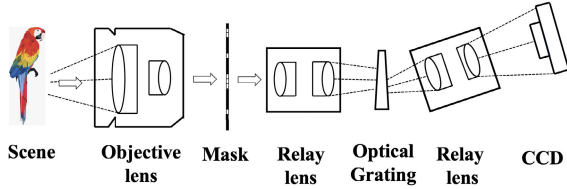


Fig. 3. Illustration of a hyperspectral SCI camera. Different from CACTI, after the temporal variant mask, the measurements go through a dispersive element like a grating to separate the measurement of different wavelengths. The mask and CCD are in the conjugate image plane of the scene.

same model except that they have different numbers of pixels and frames.

For a B -frame video tensor $\mathcal{X} \in \mathbb{R}^{n \times m \times B}$ and a tensor mask $\mathcal{M} \in \mathbb{R}^{n \times m \times B}$, the measurement frame $\mathbf{Y} \in \mathbb{R}^{n \times m}$ can be expressed as follows [6]:

$$\mathbf{Y} = \sum_{b=1}^B \mathcal{M}^{(b)} \odot \mathcal{X}^{(b)} \quad (1)$$

where $\mathcal{M}^{(b)}$ and $\mathcal{X}^{(b)}$ denote the b th frontal slice of $\mathcal{M}$ and $\mathcal{X}$, respectively, and $\odot$ denotes the elementwise (Hadamard) product, such that $\mathcal{Z}(i, j, k) = \mathcal{X}(i, j, k) \mathcal{Y}(i, j, k)$.

In hyperspectral SCI cameras [8], the expression is the same as that of video SCI cameras. We only need to change the meaning of $\mathcal{X}$ to be a spectral image tensor with the resolution of $n \times m$ in B channels of different wavelengths.

Obviously, (1) can be equivalently expressed as a linear equation

$$\mathbf{y} = \Phi \mathbf{x} \quad (2)$$

where $\mathbf{y} = \text{Vec}(\mathbf{Y}) \in \mathbb{R}^{nm}$ and $\mathbf{x} = [\text{Vec}(\mathcal{X}^{(1)}); \dots; \text{Vec}(\mathcal{X}^{(B)})] \in \mathbb{R}^{nmB}$ are the vector representations of $\mathbf{Y}$ and $\mathcal{X}$, respectively. Different from conventional compressed sensing problem, the mask Φ in (2) is a block diagonal matrix that consists of B diagonal matrices as follows [11]:

$$\Phi = [\text{diag}(\text{Vec}(\mathcal{M}^{(1)})), \dots, \text{diag}(\text{Vec}(\mathcal{M}^{(B)}))] \in \mathbb{R}^{mn \times mnB}. \quad (3)$$

C. Decoding Problem of SCI Cameras

The decoding problem (2) is an underdetermined linear inverse problem. Taking sparsity assumption into consideration, the decoding problem is equivalent to the following LASSO [34] optimization problem:

$$\min_{\mathbf{x}} \frac{1}{2} \|\Phi \mathbf{x} - \mathbf{y}\|_2^2 + \lambda \|\Psi \mathbf{x}\|_1 \quad (4)$$

where $\Psi \mathbf{x}$ is a coefficient vector in a transformation domain, $\|\cdot\|_1$ denotes the ℓ_1 -norm that imposes the sparsity on the coefficients of $\mathbf{x}$, and λ balances these two terms. According to the theory of Lasso regression, the solution of this problem regularly satisfies the sparsity requirement since it often lies on some particular coordinate axes [34].

The prerequisite for successful recovery is that the error between real $\mathbf{x}$ and the reconstructed result should be bounded by a constant. Recently, [35] provided a theorem that gives a positive answer to this requirement:

Theorem 1 [35]: Suppose that $Q \subset \mathbb{R}^{nB}$ is a compact set where n indicates the total number of pixels in one frame and B indicates the number of frames. On the other hand, $\mathbf{x}$ is a signal which $\mathbf{x} \in Q$. Assume that $\forall \mathbf{x} \in Q$, $\|\mathbf{x}\|_\infty \leq (\rho/2)$. Furthermore, assume the rate- r code achieves distortion δ on Q . Moreover, for $i = 1, \dots, B$, $\mathbf{D}_i = \text{diag}(D_{i1}, \dots, D_{in})$, and $\{D_{ij}\}_{j=1}^n \stackrel{\text{i.i.d.}}{\sim} N(0, 1)$. For $\mathbf{x} \in Q$ and $\mathbf{y} = \sum_{i=1}^B \mathbf{D}_i \mathbf{x}_i$, let $\hat{\mathbf{x}}$ denote the solution of compressible signal pursuit optimization. Assume that $\epsilon > 0$ is a free parameter, such that $\epsilon \leq (16/3)$. Then

$$\frac{1}{nB} \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 \leq \delta + \rho^2 \epsilon$$

with a probability larger than $1 - 2^{nBr+1} e^{-n((3\epsilon/32))^2}$.

Detailed proof can be found in [35]. Theorem 1 characterizes the performance of SCI recovery. It combines the rate r , distortion δ , and the number of frames B together. Also, the theorem gives the upper bound of decoding quality ϵ , which lays a solid foundation for developing decoding algorithms.

To solve (4), the most common approach is the gradient descent method while the iterative shrinkage-thresholding algorithm (ISTA) is one of the most popular gradient-based models. Fast iterative shrinkage-thresholding algorithm (FISTA) [12] is the state-of-the-art algorithm to accelerate ISTA which was first proposed in 2009. Till now, a lot of gradient-based models have been designed based on FISTA.

FISTA is realized using the following iterative steps:

$$\mathbf{r}^k = \mathbf{z}^k - \rho \Phi^T (\Phi \mathbf{z}^k - \mathbf{y}) \quad (5)$$

$$\mathbf{x}^k = \arg \min_{\mathbf{x}} \frac{1}{2} \|\mathbf{x} - \mathbf{r}^k\|_2^2 + \lambda \|\Psi \mathbf{x}\|_1 \quad (6)$$

$$t^{k+1} = \frac{1 + \sqrt{1 + 4(t^k)^2}}{2} \quad (7)$$

$$\mathbf{z}^{k+1} = \mathbf{x}^k + \frac{t^k - 1}{t^{k+1}} (\mathbf{x}^k - \mathbf{x}^{k-1}) \quad (8)$$

where $k \geq 1$. We initialize $t^1 = 1$, and $\mathbf{z}^1$ as $\mathbf{z}^1 = \mathbf{x}^0$ where the initialization of $\mathbf{x}^0$ will be proposed later. Meanwhile, ρ represents the step size, $\mathbf{r}^k$ is an auxiliary variable, $\mathbf{x}^k$ is the output of the k th iteration, and $\mathbf{z}^{k+1}$ is a new starting point for next iteration.

IV. PROPOSED Tensor FISTA-Net

In this section, we mainly discuss our Tensor FISTA-Net and the acceleration techniques we use. We will convert the matrix-form calculations into those of tensor form and then unfold the iteration steps into a DNN. We will then introduce the design details of the proposed Tensor FISTA-Net. Afterward, the loss function and initialization process will be presented. Finally, we introduce a plug-and-play method to update the ℓ_1 -norm penalty parameter dynamically.

A. Acceleration Techniques

To meet the requirement of real-time in real-life devices, we adopt some accelerating techniques to decrease the computational complexity further. These techniques mainly include the following.

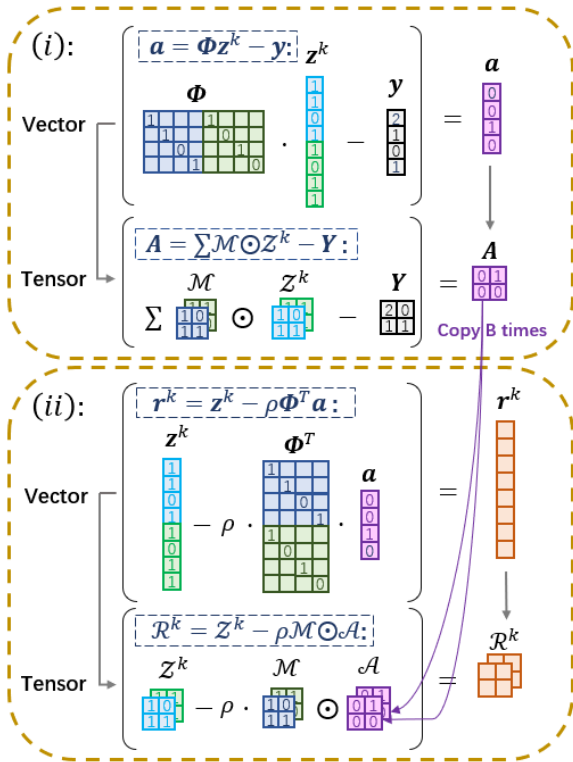


Fig. 4. Illustration of tensor calculations of $\mathcal{R}^k$. $\mathbf{A}$ in the first step is an intermediate result. In the second step, $\mathbf{A}$ is copied B times along the third dimension to form tensor $\mathcal{A}$.

- 1) We unfold the matrix-form calculations into tensor-form calculations, which brings an improvement in computational complexity from $O(m^2n^2B)$ to $O(mnB)$.
- 2) We split the training and decoding procedure into discrete parts, and therefore only one forward operation is performed when decoding a compressed frame.
- 3) We apply the accelerating linear algebra (XLA) compiler in TensorFlow to compile and execute the network in a special arithmetic kernel, accelerating the calculations.

B. Tensor FISTA Algorithm

FISTA uses a specific linear combination of two previous iteration results to do the latest iteration while ISTA only uses one. Using ISTA, the convergence rate of $F(x^k) - F(x^*)$ is only $O((1/k))$. However, in FISTA [12] this convergence rate could reach $O((1/k^2))$, which is a large improvement toward real-time decoding. However, we cannot directly apply FISTA to the decoding problem of SCI cameras since FISTA (5)–(8) still requires a lot of computation in matrix forms. So we are motivated to design a tensor FISTA algorithm.

Specifically, consider $\mathcal{Z}^k$, $\mathcal{R}^k$ and $\mathcal{X}^k$ as the tensor form of $\mathbf{z}^k$, $\mathbf{r}^k$ and $\mathbf{x}^k$, respectively. Iterative steps in (5)–(8) are considered as four modules: t^k , $\mathcal{Z}^k$, $\mathcal{R}^k$, and $\mathcal{X}^k$ in our proposed tensor FISTA algorithm. Different from the order in (5)–(8), we calculate t^k and $\mathcal{Z}^k$ before $\mathcal{R}^k$ and $\mathcal{X}^k$ in each phase for simplicity.

Fig. 4 is a brief illustration of the transformation between matrix-form and tensor-form calculations, taking the calculation of $\mathcal{R}^k$ as an example. Therefore, the proposed tensor FISTA algorithm can be concluded by the following

expressions: ($k \geq 1$ represents the number of iterations)

$$t^1 = 1 \quad (9)$$

$$\mathbf{z}^1 = \mathbf{x}^0 \quad (10)$$

$$\mathcal{R}^k = \mathcal{Z}^k - \rho \mathcal{M} \odot \mathbf{A} \quad (11)$$

$$\mathcal{A}^{b_0} = \sum_{b=1}^B \mathcal{M}^b \odot \mathcal{Z}^{k(b)} - \mathbf{Y}, \quad b_0 = 1, \dots, B \quad (12)$$

$$\mathcal{X}^k = \arg \min_{\mathcal{X}} \frac{1}{2} \|\mathcal{X} - \mathcal{R}^k\|_F^2 + \lambda \sum_{i=1}^2 \|\mathcal{F}(\mathcal{X})\|_1 \quad (13)$$

$$t^{k+1} = \frac{1 + \sqrt{1 + 4(t^k)^2}}{2} \quad (14)$$

$$\mathcal{Z}^{k+1} = \mathcal{X}^k + \frac{t^k - 1}{t^{k+1}} (\mathcal{X}^k - \mathcal{X}^{k-1}). \quad (15)$$

In (13), we change the orthogonal transformation matrix Ψ into an orthogonal transformation function $\mathcal{F}$ which acts on the tensor-form variable $\mathcal{X}$.

Meanwhile, in (13), the definition of ℓ_1 -norm of a tensor is defined as follows:

$$\|\mathcal{X}\|_1 = \sum_{1 \leq i_1 \leq n, 1 \leq i_2 \leq m, 1 \leq i_3 \leq B} |\mathcal{X}_{i_1 i_2 i_3}| \quad (16)$$

which is the sum of the elements' absolute values, a bit different from the definition of the matrix's ℓ_1 norm.

Unfolding matrices and vectors into tensors is our critical acceleration technique. Taking the operations in Fig. 4 as an example, Φ is of the size $mn \times mnB$ for a generalized model. Generally, the matrix form calculations will have the computation complexity of $O(m^2n^2B)$. However, the tensor form of $\mathcal{M}$ has the size of $\sqrt{mn} \times \sqrt{mn} \times B$, thus having the computational complexity of $O(mnB)$. Therefore, transforming the calculation of FISTA from matrix form to tensor form prominently decreases the computational complexity, proving that our Tensor FISTA-Net is more suitable for real-time decoders of SCI cameras than the state-of-the-art algorithms.

C. Tensor FISTA-Net

The basic idea of Tensor FISTA-Net is to unfold (11)–(15) into a DNN with a fixed number of phases, where each phase corresponds to one iteration. We are motivated to design the Tensor FISTA-Net for the following four reasons.

- 1) Temporal correlations within video frames. Video frames are similar to each other, and therefore by stacking them into a tensor in the third dimension, we are able to learn the temporal correlations within them through multichannel 2-D convolution, which works better in capturing the information than that through single-channel 2-D convolution in a matrix form.
- 2) Incorrect features extracted in matrix form. In matrix form, if we concatenate one B -frame video into a big image with size $nB \times m$, convolution kernels may extract incorrect spatial information at the borders between two frames. This weakness can be avoided in tensor form.
- 3) Reduction in running time and memory consumption through tensor calculations. Due to the special structure

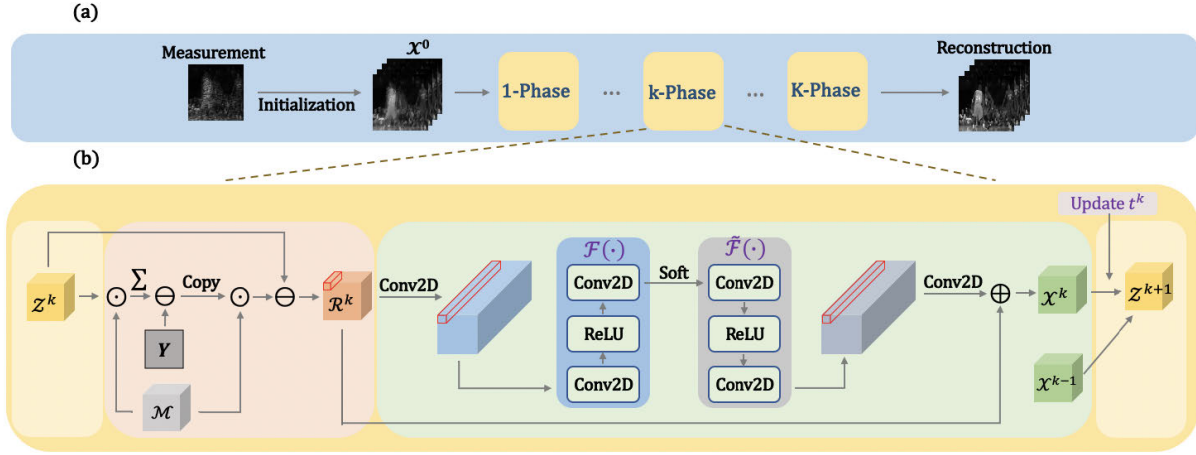


Fig. 5. Structure of the proposed Tensor FISTA-Net. The upper part shows the data flow in Tensor FISTA-Net, which contains K phases. The bottom part shows the detailed structure of a single phase. We start by updating $\mathcal{R}^k$ first, and then updating t^k and $\mathcal{X}^k$ using CNN. Finally, we update $\mathcal{Z}^k$. Gray arrows represent the data flow.

of Φ in (3), updating (5) needs to multiply big matrices Φ^T and Φ . However, abundant time and memory will be saved if we update (5) in tensor form.

- 4) Compared with *Tensor ADMM-Net* [15], our Tensor FISTA-Net is based on the sparse constraint of the FISTA algorithm, while *Tensor ADMM-Net* is based on the low-rank constraint of the ADMM algorithm. Besides, Tensor ADMM-Net requires additional transformations to the frequency domain, which brings additional computation complexity in the unfolding process.
- 5) Our Tensor FISTA-Net is based on our proposed tensor-form FISTA algorithm. Therefore, it no longer needs vector- and matrix-form calculations compared with *Tensor ADMM-Net*.

Fig. 5 illustrates the whole framework of Tensor FISTA-Net and the specific design in a phase. In the proposed Tensor FISTA-Net, measurement frames collected by CCD first undergo an initialization process to provide initial values for training and validating the model. Then the network goes through a K -phase training period to get the decoding model. After that, a brand new measurement is provided and reconstructed image frames are created from the model trained by the neural network.

To elaborate the structure of Tensor FISTA-Net, we will first propose the update method of $\mathcal{X}^k$ and t^k using DNNs.

Module t^k : In (14) and (15), we carefully set the step size for t^k to find a relatively optimized trajectory of gradient descent method. However, since DNNs have great learning capabilities, we make t^k for $k \in [K]$ as learnable variables to improve the flexibility of the proposed Tensor FISTA-Net, also to find a dynamic step size.

Module $\mathcal{X}^k$: (13) uses an orthogonal and also sparse transformation as $\mathcal{F}$. However, we aim to find the sparsest transformation domain rather than using a preset one. Inspired by [36] and [37], we take the temporal correlations into consideration and design the general transformation based on tensor form.

Consider the great representation power of convolution neural network (CNN) [38] and its universal approximation property [39], we use convolution and activation layers to

represent the transformation function denoted by $\mathcal{F}(\cdot)$, which is supposed to learn the sparsest representation of video frames. Specifically, $\mathcal{F}(\cdot)$ contains two multichannel 2-D convolution layers split by one ReLU activation layer. We denote it as $\mathcal{F}(\mathcal{X}) = \text{Conv2-D}(\text{ReLU}(\text{Conv2-D}(\mathcal{X})))$.

After replacing Ψ with $\mathcal{F}(\cdot)$, the tensor form of (6) should be as follows:

$$\mathcal{X}^k = \arg \min_{\mathcal{X}} \frac{1}{2} \|\mathcal{X} - \mathcal{R}^k\|_F^2 + \lambda \|\mathcal{F}(\mathcal{X})\|_1. \quad (17)$$

$\mathcal{F}(\mathbf{x})$ is equivalent to performing a matrix multiplication to $\mathbf{x}$, i.e., $\mathcal{F}(\mathbf{x}) = \mathbf{D}\mathbf{x}$, where $\mathbf{D}$ is an orthonormal matrix which consists of all the basis vectors in transformation domain. Therefore, the spectral norm of $\mathbf{D}$ is 1, which indicates that $\|\mathbf{D}\mathbf{a} - \mathbf{D}\mathbf{b}\|_2 = \|\mathbf{a} - \mathbf{b}\|_2$ holds for all the matrices $\mathbf{a}$ and $\mathbf{b}$.

With the above property, $\|\mathbf{x} - \mathbf{r}^k\|_2^2 = \|\mathbf{D}\mathbf{x} - \mathbf{D}\mathbf{r}^k\|_2^2 = \|\mathcal{F}(\mathbf{x}) - \mathcal{F}(\mathbf{r}^k)\|_2^2$, since $\|\cdot\|_2$ denotes the ℓ_2 -norm, the square root of the sum of square of all the elements. The property also holds when extended to tensor form, so we replace $\|\mathcal{F}(\mathbf{x}) - \mathcal{F}(\mathbf{r}^k)\|_2^2$ by $\|\mathcal{F}(\mathcal{X}) - \mathcal{F}(\mathcal{R}^k)\|_F^2$ and (17) becomes

$$\mathcal{X}^k = \arg \min_{\mathcal{X}} \frac{1}{2} \|\mathcal{F}(\mathcal{X}) - \mathcal{F}(\mathcal{R}^k)\|_F^2 + \lambda \|\mathcal{F}(\mathcal{X})\|_1. \quad (18)$$

Soft-thresholding operator [40] is an elementwise operator when applied to a tensor. Lemma 1 proves that the soft-thresholding operator and the optimization problem (18) are equivalent.

Lemma 1 [40]: Given $\mathcal{R} \in \mathbb{R}^{n \times m \times B}$ and $\mathcal{X} \in \mathbb{R}^{n \times m \times B}$ is the variable to be optimized, Then the optimization problem

$$\mathcal{X}^* = \arg \min_{\mathcal{X}} \frac{1}{2} \|\mathcal{X} - \mathcal{R}\|_F + \lambda \|\mathcal{X}\|_1 \quad (19)$$

has the following closed-form solution:

$$\mathcal{X}_{ijk}^* = \begin{cases} \mathcal{R}_{ijk} - \lambda, & \mathcal{R}_{ijk} > \lambda \\ \mathcal{R}_{ijk} + \lambda, & \mathcal{R}_{ijk} < -\lambda \\ 0, & -\lambda \leq \mathcal{R}_{ijk} \leq \lambda \end{cases} \quad (20)$$

for all $i \in [n]$, $j \in [m]$ and $k \in [B]$.

The transformation between tensor-form and scalar-form soft-thresholding operators is trivial

$$\frac{1}{2}\|\mathcal{X} - \mathcal{R}\|_2 + \lambda\|\mathcal{X}\|_1 = \sum_{ijk} \frac{1}{2}|\mathcal{X}_{ijk} - \mathcal{R}_{ijk}|^2 + \lambda|\mathcal{X}_{ijk}|. \quad (21)$$

The soft-thresholding operator is denoted by $\text{soft}(\mathcal{R}, \lambda)$.

Therefore, according to Lemma 1 and [40], we adopt it to obtain a closed-form solution of $\mathcal{F}(\mathcal{X}^k)$ (18)

$$\mathcal{F}(\mathcal{X}^k) = \text{soft}(\mathcal{F}(\mathcal{R}^k), \lambda). \quad (22)$$

This transformation helps change the original optimization problem which is very difficult to express especially in the tensor form to an elementwise production. Therefore, a lot of reconstruction time could be saved.

To get $\mathcal{X}^k$ from $\mathcal{F}(\mathcal{X}^k)$, we introduce an inverse transformation function $\tilde{\mathcal{F}}(\cdot)$ to inverse the transformation function $\mathcal{F}(\cdot)$. $\tilde{\mathcal{F}}(\cdot)$ is supposed to have the ability of inversion, i.e. $\tilde{\mathcal{F}}(\mathcal{F}(\mathcal{X})) = \mathcal{X}$. Therefore, the inverse function $\tilde{\mathcal{F}}(\cdot)$ is realized using the symmetric structure of $\mathcal{F}(\cdot)$, so it can be written as $\tilde{\mathcal{F}}(\mathcal{X}) = \text{Conv2-D}(\text{ReLU}(\text{Conv2-D}(\mathcal{X})))$, and the only difference between $\mathcal{F}(\cdot)$ and $\tilde{\mathcal{F}}(\cdot)$ is the convolution kernels.

Then we adopt the inverse transformation function $\tilde{\mathcal{F}}(\cdot)$ and obtain the closed-form solution of $\mathcal{X}^k$

$$\mathcal{X}^k = \tilde{\mathcal{F}}(\text{soft}(\mathcal{F}(\mathcal{R}^k), \lambda)). \quad (23)$$

Finally, inspired by the Residual Network [41], we add a shortcut structure and the solution of $\mathcal{X}^k$ is as follows:

$$\mathcal{X}^k = \mathcal{R}^k + \tilde{\mathcal{F}}(\text{soft}(\mathcal{F}(\mathcal{R}^k), \lambda)). \quad (24)$$

The summarized algorithm is given in Algorithm 1. In the algorithm, we first initialize the data for training and validation, along with the auxiliary variables. Then in every iteration we first update $\mathcal{R}^k$, and then update t^k and $\mathcal{X}^k$ using CNN. Finally, we update $\mathcal{Z}^k$ and calculate the loss function. If the loss is less than a threshold or the number of iterations reaches the maximum, we end the training process.

D. Design of $\mathcal{F}(\cdot)$ and $\tilde{\mathcal{F}}(\cdot)$

In Tensor FISTA-Net, convolution layers perform the linear transformation for video and spectral frames, and activation layers introduce nonlinearity to the transformation and impose its sparsity. Specifically, we use two convolution layers split by the ReLU layer to serve as the transformation structure, denoted by $\mathcal{F}(\mathcal{X}) = \mathcal{C}_2(\mathcal{R}(\mathcal{C}_1(\mathcal{X})))$, where $\mathcal{C}_1$ and $\mathcal{C}_2$ denote two convolution layers with different convolution kernels, respectively. Similarly, the inverse transformation function $\tilde{\mathcal{F}}(\cdot)$ can be expressed as $\tilde{\mathcal{F}}(\mathcal{X}) = \tilde{\mathcal{C}}_2(\mathcal{R}(\tilde{\mathcal{C}}_1(\mathcal{X})))$, where $\tilde{\mathcal{C}}_1$ and $\tilde{\mathcal{C}}_2$ denote two convolution layers with different convolution kernels, respectively.

Considering the tight temporal correlations among the frontal slices of the video and spectral tensors, we use multichannel 2-D convolution because it extracts information among temporal sequences and learns a sparse representation of the video tensors. For simplicity, we use the same kernel size ($3 \times 3 \times B$) in all the convolution layers.

Algorithm 1 Tensor FISTA-Net for SCI Cameras

Input: $\Phi, y, k_{\max}, \epsilon$

Output: Reconstruction of X

Initialization: $\mathbf{M} = \sum_{b=1}^B \mathcal{M}^{(b)}, \mathcal{X}^{(0)} = \mathbf{Y} \oslash \mathbf{M};$
 $z^1 = x^0, t^1 = 1, k = 0, L = \mathcal{L}(\mathbf{0}, \mathcal{X}^0);$

while $k \leq k_{\max}$ and $L > \epsilon$ **do**

$k = k + 1;$

$\mathcal{A}^{b_0} = \sum_{b=1}^B \mathcal{M}^{(b)} \odot \mathcal{Z}^{k(b)} - \mathbf{Y}, b_0 = 1, \dots, B;$

$\mathcal{R}^k = \mathcal{Z}^k - \rho \mathcal{M} \odot \mathcal{A};$

 Update $\mathcal{X}^k$: $\mathcal{X}^k = \mathcal{R}^k + \tilde{\mathcal{F}}(\text{soft}(\mathcal{F}(\mathcal{R}^k), \lambda));$

 Update t^k to $t^{k+1};$

$\mathcal{Z}^{k+1} = \mathcal{X}^k + t^{k+1}(\mathcal{X}^k - \mathcal{X}^{k-1});$

$L = \mathcal{L}(\mathcal{X}^k, \mathcal{X}^0);$

end

In addition, to improve the representation ability, we use an extra convolution layer to increase the number of channels from B to 64 before the transformation function $\mathcal{F}(\cdot)$ and use another convolution layer to reduce the number of channels from 64 to B after the inverse transformation function $\tilde{\mathcal{F}}(\cdot)$. Specifically, we keep the input and output channels as 64 in $\mathcal{F}(\cdot)$ and $\tilde{\mathcal{F}}(\cdot)$.

E. Initialization

A proper initialization helps reduce the training time significantly. Naturally, taking the special structure of the mask Φ in (2) into consideration, we initialize each frontal slice of $\mathcal{X}^0$ as the measurement frame divided by the sum of mask in the third dimension. Consider $\mathbf{M} \in \mathbb{R}^{n \times m}$ to be the sum of mask in third dimension, $\mathcal{X}^0 \in \mathbb{R}^{n \times m \times B}$ can be initialized as follows:

$$\mathbf{M} = \sum_{b=1}^B \mathcal{M}^{(b)} \quad (25)$$

$$\mathcal{X}^{0(b)} = \mathbf{Y} \oslash \mathbf{M}, \text{ for } b \in [B] \quad (26)$$

where $\oslash$ denotes elementwise division between two matrices.

F. Loss Function

Three constraints should be taken into consideration in Tensor FISTA-Net.

- 1) The fidelity of the reconstructed frames.
- 2) The accuracy of the inverse function.
- 3) The sparsity of video frame representation in the learned domain.

Assume there are K phases in total, $\mathcal{X}^k \in \mathbb{R}^{n \times m \times B}$ for $k \in [K]$ is the output of the k th phase, the three constraints can be expressed as follows:

$$\mathcal{L}_{\text{fidelity}} = \|\mathcal{X}^K - \mathcal{X}\|_F^2 \quad (27)$$

$$\mathcal{L}_{\text{inversion}} = \frac{1}{K} \sum_{k=1}^K \|\tilde{\mathcal{F}}(\mathcal{F}(\mathcal{X}^k)) - \mathcal{X}^k\|_F^2 \quad (28)$$

$$\mathcal{L}_{\text{sparsity}} = \frac{1}{K} \sum_{k=1}^K \|\mathcal{F}(\mathcal{X}^k)\|_1. \quad (29)$$

The loss function is a weighted sum of these three terms

$$\mathcal{L} = \alpha \mathcal{L}_{\text{fidelity}} + \beta \mathcal{L}_{\text{inversion}} + \gamma \mathcal{L}_{\text{sparsity}} \quad (30)$$

where α , β , and γ are coefficients to balance the three terms. By default, we set $\alpha = 1$, $\beta = 0.01$, and $\gamma = 0.001$.

G. Plug-and-Play Method for Tensor FISTA-Net

Inspired by the PnP-GAP and PnP-ADMM algorithms [24], we find that the plug-and-play method can also be applied to our Tensor FISTA-Net to dynamically update the sparsity penalty parameter λ , instead of directly setting it to a constant. Therefore, the sparsity penalty could be dynamically optimized, which helps improve the quality of reconstruction results.

Recalling the third step (13) in the optimization problem of the original tensor FISTA algorithm without the aid of DNN

$$\mathcal{X}^k = \arg \min_{\mathcal{X}} \frac{1}{2} \|\mathcal{X} - \mathcal{R}\|_F + \lambda \|\mathcal{X}\|_1. \quad (31)$$

In our Tensor FISTA-Net, we apply the soft-thresholding operator to transform the solution of this optimization problem into a linear form. Fortunately, it does not affect the insertion of the plug-and-play method. Here, we apply the dynamic update scheme

$$\Delta_{k+1} = \|\mathcal{X}^{k+1} - \mathcal{X}^k\|_2. \quad (32)$$

- 1) If $\Delta_{k+1} > \mu \Delta_k$, then $\lambda_{k+1} = \alpha \lambda_k$.
- 2) Otherwise, $\lambda_{k+1} = \lambda_k$.

where μ and α are constant parameters to be set previously. Then the optimization problem will become

$$\mathcal{X}^k = \arg \min_{\mathcal{X}} \frac{1}{2} \|\mathcal{X} - \mathcal{R}\|_F + \lambda_{k-1} \|\mathcal{X}\|_1. \quad (33)$$

We set $\lambda_0 = 0.125$, a little bigger than the current value, and set $\mu = \alpha = 0.95$ for the aim of accuracy–penalty tradeoff. We reserve the previous version of the fixed penalty to observe whether there is any improvement after inserting the plug-and-play method in our Tensor FISTA-Net.

V. PERFORMANCE EVALUATION ON VIDEO SCI CAMERAS

Video SCI cameras are widely used to capture multi-frame video images in real-life cases such as sports events and car crashing experiments. To meet the requirement of real-time in those real-world applications, a real-time decoder is recommended. Therefore, we evaluate the performance of Tensor FISTA-Net on different synthetic and real video SCI datasets. First, we describe the datasets and the implementation details. Then, we carry out the experiment and compare the performance with other algorithms. After that, we will present the decoded images generated by Tensor FISTA-Net and other algorithms and compare some detailed areas in those images.

A. Datasets

We use the same synthetic training videos NBA, Central Park Aerial, and Vehicle Crashing Tests as in [15]. We resize the video frames into 256×256 through downsampling and extract the luminance component to make the training datasets; each training dataset contains 8000 frames, i.e., 1000 measurements. The synthetic datasets used in our *Tensor-FISTA-Net* are generated according to the physical structure and coding patterns of the video SCI cameras, and they are unrelated to the real testing videos. Before the reconstruction process, the modulation patterns are precalibrated by illuminating the masked DMD with uniform light and capturing the patterns of the camera's mask [42]. Furthermore, we manually add zero-mean Gaussian noise with variance scale $\approx 10^{-3}$ [9] to the synthetic dataset, which is large enough to ignore the noise in the image capture process.

For video SCI cameras, we evaluate the performance of the proposed Tensor FISTA-Net on three different synthetic datasets: Kobe [9], Park [15], and Vehicle [15]. Each testing dataset consists of 32 frames of size 256×256 and $B = 8$, i.e., four measurements. We also evaluate the performance of Tensor FISTA-Net on two different real datasets: *Hammer* and *Wheel* and compare its results with those of previous algorithms.

To compare with previous algorithms, we use the same synthetic mask following the settings in [11]. Peak signal-to-noise ratio (PSNR), structural similarity index (SSIM) [43], and reconstruction time are the performance metrics in comparison.

B. Implementation Details

We use TensorFlow to implement our algorithm and carry out all the experiments on an NVIDIA DGX server with a Tesla A100-SXM4 GPU (40-GB device memory). In reality, the upstream and downstream speeds of the network device exceed 10 MB/s, which is sufficient to support the real-time transmission of reconstructed model information.

We set the number of phases as 10, the learning rate as 0.00025, and the running epoch as 500. Adam optimizer [44] is used to minimize the training loss. The code is available at <https://github.com/Chesterhuang1999/Tensor-FISTA-Net>.

C. Comparison With State-of-the-Art Algorithms

GAP-TV [18] applied the idea of total variance minimization under the generalized alternating projection (GAP) to the decoding problem. It can be used as a benchmark for the decoding results because it is simple and fast.

DeSCI [11] exploited rank minimization for nonlocal patches and obtained the best decoding results among those optimization-based algorithms.

Tensor ADMM-Net [15] is the first network-based algorithm that adopts deep unfolding. It works by unfolding a tensor ADMM algorithm into a DNN, which appears to be similar to Tensor FISTA-Net. However, ADMM may be inferior to FISTA in solving linear inverse problems since the convergence rate of ADMM is slower than that of FISTA.

TABLE I
AVERAGE PSNR (dB) AND SSIM ON DIFFERENT DATASETS

Algorithm	Kobe	Park	Vehicle
Tensor FISTA-Net (Ours)	31.41/0.92	26.67/ 0.92	26.46/0.89
GAP-TV [18]	26.45/0.84	24.53/0.84	22.85/0.77
DeSCI [11]	33.25/0.95	24.95/0.80	21.16/0.70
Tensor ADMM-Net [15]	30.15/0.89	26.85/0.86	23.62/0.78
SeSCI [20]	30.16/0.91	25.41/0.82	24.81/0.81
PnP-FFDNet [24]	30.33/0.92	24.67/0.83	24.29/0.82
BIRNAT [28]	32.71/0.95	28.99/0.91	27.84/0.92

TABLE II
AVERAGE RECONSTRUCTION TIME PER MEASUREMENT (SECONDS) ON DIFFERENT DATASETS

Algorithm	Kobe	Park	Vehicle
Tensor FISTA-Net (Ours)	0.18	0.17	0.17
GAP-TV [18]	7.9	6.9	7.2
DeSCI [11]	6872.9	6915.8	6823.5
Tensor ADMM-Net [15]	1.9	2.4	2.1
SeSCI [20]	55.0	62.0	60.6
PnP-FFDNet [24]	1.8	1.8	1.8
BIRNAT [28]	0.16	0.16	0.16

Therefore, *Tensor ADMM-Net* is expected to use more phases (also network layers) than *Tensor FISTA-Net*.

PnP-FFDNet [24] uses the PnP-GAP algorithm and FFDNet as an additional denoiser. Also, with the aid of DNNs, PnP-FFDNet reaches very short reconstruction time, which could be viewed as a promising candidate for real-time decoding of SCI cameras.

BIRNAT [28] first uses convolutional neural networks with self-attention to construct a reference frame that extracts the nonlocal dependencies. Then it uses recurrent neural networks (RNNs) to exploit the temporal correlations between adjacent frames. Furthermore, dual-stage RNN and adversarial training are jointly applied to improve the reconstruction results further.

To compare the decoding accuracy of different algorithms, we calculate PSNR and SSIM of the experimental results. Table I illustrates the reconstruction accuracy of three synthetic datasets using *Tensor FISTA-Net* and other algorithms. On the *Kobe* dataset, *DeSCI* provides the best reconstruction results. Our *Tensor FISTA-Net* provides the best results on the *Park* dataset (0.79 dB in PSNR and 0.02 in SSIM higher than the state-of-the-art algorithm PnP-FFDNet and *Tensor ADMM-Net*) and *Vehicle* dataset (2.84 dB in PSNR and 0.11 in SSIM higher than the state-of-the-art algorithm).

To validate the decoding results, we show the reconstructed frames of three datasets using different algorithms in Fig 6. Although our *Tensor FISTA-Net* does not achieve the best results in *Kobe*, it still provides better results than other algorithms except for *DeSCI*. From the decoded frames of the *Park* and *Vehicle* datasets, we observe that *DeSCI* cannot exquisitely reconstruct the details of the videos, especially when a small area contains many details like the branches in the *Park* dataset and edges of cars in the *Vehicle* dataset (marked by yellow square). Moreover, we note that *DeSCI* suffers from being over-smooth because it smooths out tiny details. This indicates that if similar patches cannot be found in video frames, the reconstruction quality of *DeSCI* will be limited. *Tensor ADMM-Net* provides better results than *DeSCI* in the *Park* and *Vehicle* datasets, but it still suffers from

blur and noise compared with our *Tensor FISTA-Net* such as tower in the *Park* dataset (marked by red square) and edges of cars in the *Vehicle* dataset (marked by yellow square).

Running Time Analysis: For the time complexity analysis of different algorithms, we record the running time on three synthetic datasets of different algorithms. Table II shows that the decoding speed of *Tensor FISTA-Net* is over four times faster than *GAP-TV* and even over 4K times faster than *DeSCI* in average. In fact, *Tensor FISTA-Net* only takes 0.5 s to reconstruct a measurement in our *Kobe* dataset, which is even faster than *Tensor ADMM-Net* too. This is because DNN approaches trade training time for decoding time. Compared with the counterpart of real-time decoder candidates, PnP-FFDNet, our *Tensor FISTA-Net* still outperforms, by a 1-s advantage over PnP-FFDNet. Since the physical models of SCI cameras are fixed, the model could be trained at the cloud before reconstruction. Moreover, with the aid of GPU, the training time of our *Tensor FISTA-Net* has been pulled down to 8 h on average.

In addition, the size of our neural network model is less than 12 MB, so it is suitable for real-time IoT devices with a small memory such as drones. Table III is an integrated table consisting of implementing efficiencies of different algorithms, evaluated from various perspectives. The very short reconstruction time of our *Tensor FISTA-Net* indicates that it is the optimal choice for the real-time decoder of SCI cameras.

D. Real SCI Datasets

In this section, we apply the proposed *Tensor FISTA-Net* on real SCI datasets *Wheel* (grayscale, $B = 14$) and *Hammer* (bayer RGB, $B = 22$) captured by a video SCI camera [6].

In *Wheel* videos, a chopper wheel is rotating in a fixed high speed so that SCI camera is suitable for capturing its high-speed motions.

In *Hammer* videos, a hammer that swings back and forth like a pendulum hits a plastic apple and knocks it down. SCI video cameras capture the detailed motions of both hammer and apple.

These two real datasets compress more video frames into one measurement than synthetic datasets, which is a greater challenge for us. Figs. 7 and 8 show ten selected reconstruction frames of the *Wheel* and *Hammer* datasets, respectively.

From the decoded frames, we observe that the proposed *Tensor FISTA-Net* provides clearer edges and less noise in the *Wheel* dataset. In the *Hammer* dataset, *Tensor FISTA-Net* provides the decoded frames with less phantom than *DeSCI* and *Tensor ADMM-Net* and much less noise than *GAP-TV*.

VI. PERFORMANCE EVALUATION OF HYPERSPECTRAL SCI CAMERAS

Hyperspectral imaging (HSI) is an important visual modality with growing interest and a bigger range of applications. It provides more information in certain scenarios than normal RGB images. *Tensor FISTA-Net* provides a new real-time approach to decode hyperspectral images. This section shows the performance of *Tensor FISTA-Net* on different datasets of hyperspectral SCI cameras. In addition to the approaches

TABLE III
IMPLEMENTING EFFICIENCIES OF DIFFERENT ALGORITHMS. GAP-TV, DESCI, AND SeSCI ARE NOT NEURAL-NETWORK-BASED MODELS, THEREFORE DO NOT HAVE DATA OF TRAINING TIME

Methods	training (hr)	decoding (s)	Model size (MB)	PSNR (dB)	frame rate (frames/s)
FISTA-Net (Ours)	4.5	0.18	12	31.41	44.4
GAP-TV [18]	/	7.9	/	26.45	1.0
DeSCI [11]	/	6872.9	69.6	33.25	0.0011
ADMM-Net [15]	6.9	1.9	29	30.15	4.2
SeSCI [20]	/	55.0	295.7	30.16	0.145
BIRNAT [28]	72	0.16	15	32.71	0.95

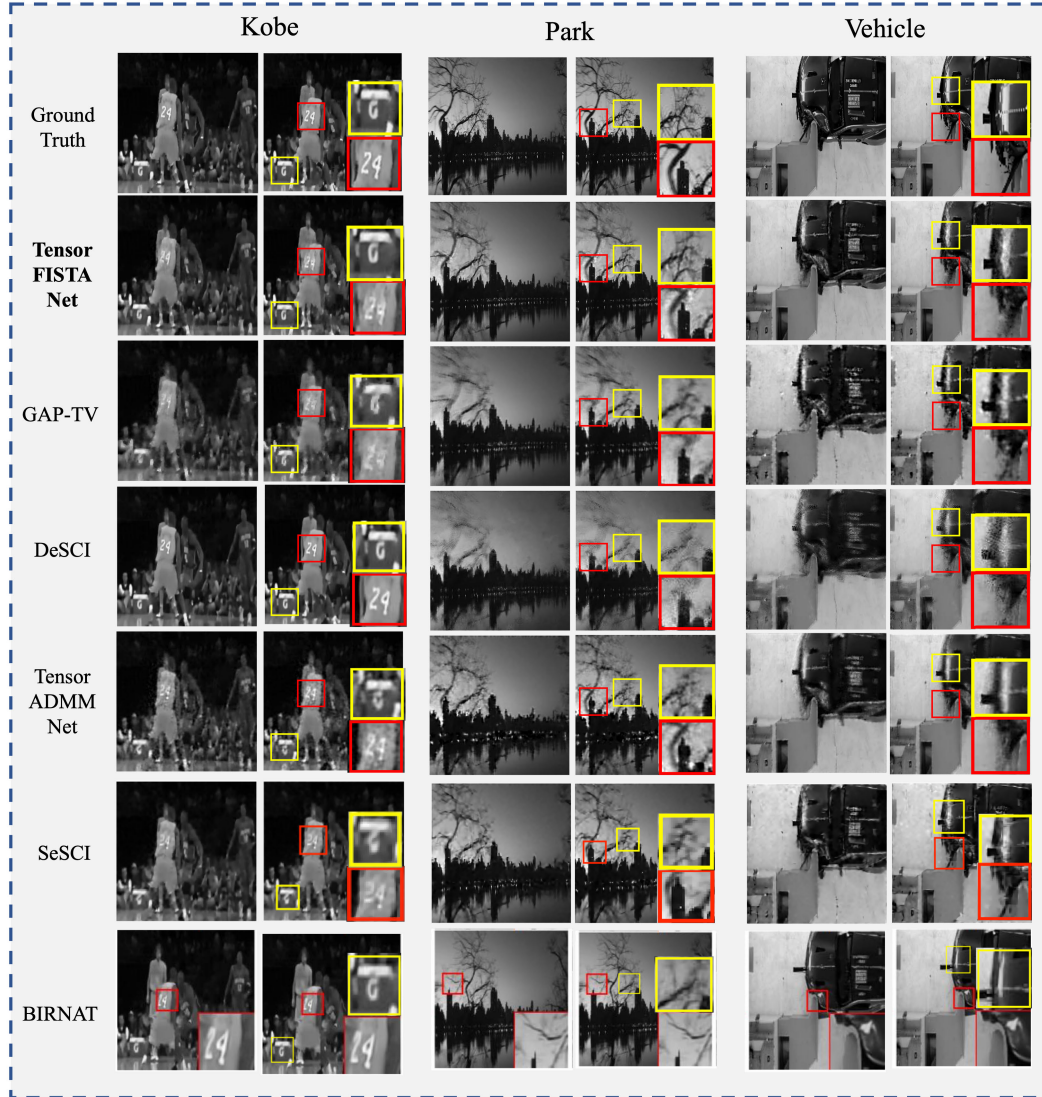


Fig. 6. On Kobe, Park, and Vehicle datasets (256×256 , $B = 8$): Two selected ground truth and decoded frames from each dataset and detailed enlarged views of certain areas. Tensor FISTA-Net performs best in Park and Vehicle, showing the least noise and blur in the decoded images. In Kobe, DeSCI performs best but only outperforms Tensor FISTA-Net very slightly. DeSCI also suffers from noisy and blurry images in Park and Vehicle.

used to test video SCI cameras, we apply spectral density correlation as an alternative tool to analyze the decoding results of different algorithms on hyperspectral images.

A. Datasets

We choose the same training and testing datasets as those in [23] and [45], which can be directly downloaded from <http://icvl.cs.bgu.ac.il/hyperspectral/>. The original dataset of [45] is collected by a complete set of the multichannel spectral camera. The training dataset has 150 images while

the test dataset has ten images. The mask provided in [23] is $256 \times 256 \times 24$. However, the original datasets of [45] have a uniform image size of $1392 \times 1300 \times 31$, which consists of 31 images of different wavelengths, ranging from 400 to 700 nm with a 10-nm interval, while the testing data and the mask have the size of $256 \times 256 \times 24$, which have 24 particular channels. Therefore, to use the provided mask, we first perform a cut of original images to generate a brand new set of training data with size 256×256 , and then apply interpolation to transform the data from 31 channels to 24 channels.

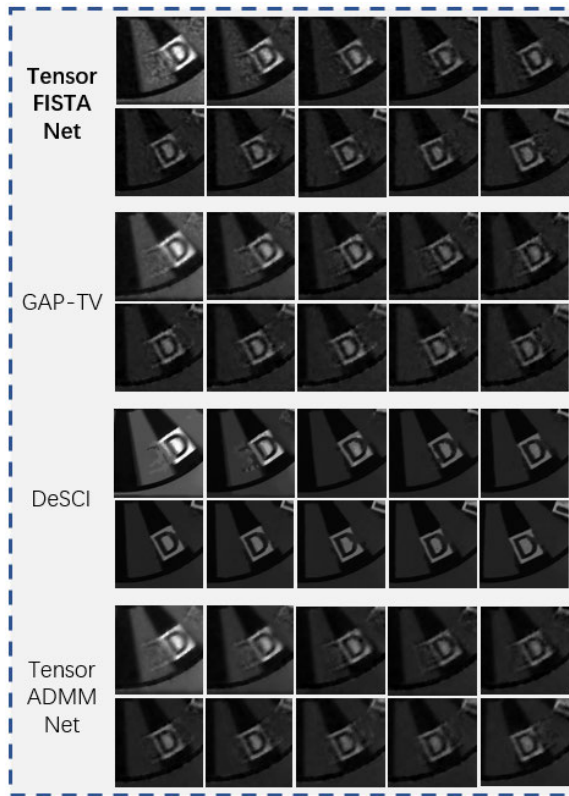


Fig. 7. On the Wheel dataset (256×256 , $B = 14$): a Chopper Wheel is rotating at high speed. Ten real gray SCI measurement reconstruction frames are selected to illustrate the decoding results.

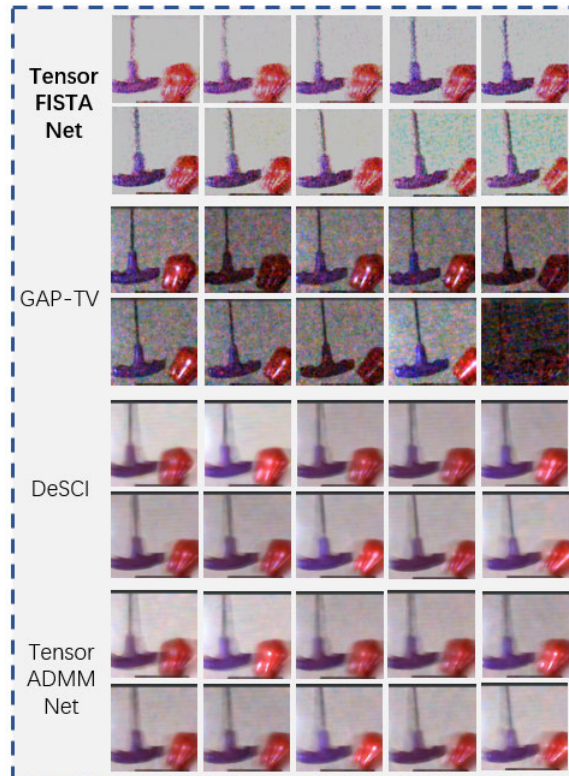


Fig. 8. On the Hammer dataset (512×512 , $B = 22$): A hammer swings like a pendulum and hits a plastic apple. Ten real bayer RGB SCI measurement reconstruction frames are selected to illustrate the decoding results.

Fortunately, Miao et al. [23] have already finished the data interpolation process and we can simply use their images.

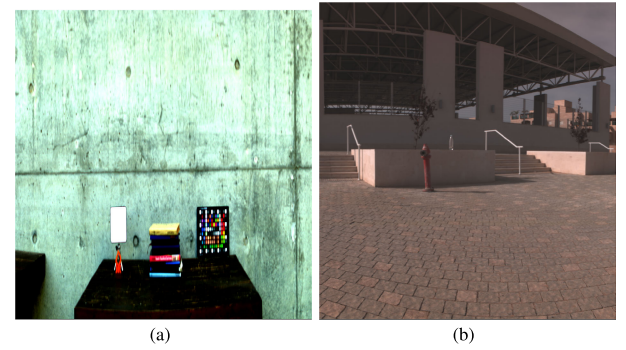


Fig. 9. Two training images selected from the dataset. (a) 101th image. (b) 102nd image.

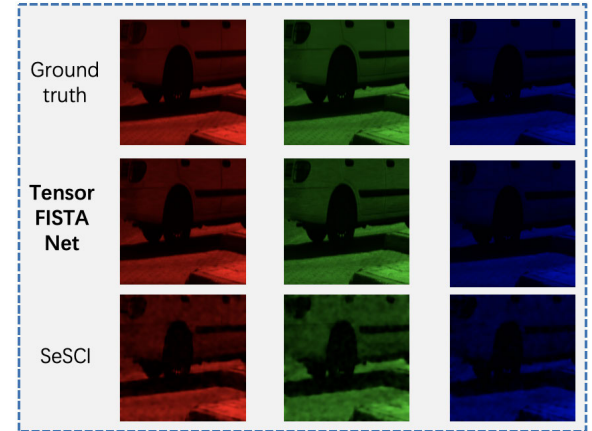


Fig. 10. Selected reconstruction images in RGB channels. It shows that Tensor FISTA-Net performs better than SeSCI more clearly than those integrated images.

Fig. 9 shows two images selected from the database: One shows a square table leaning against a stone wall, with books and a board full of colored blocks on it. Another one illustrates the edge of a stadium.

The data are used to train the proposed Tensor FISTA-Net and check the quality of the reconstruction process. To satisfy the demand of data format in different algorithms, for example, Tensor FISTA-Net needs a 4-D tensor which is the same as [23] while [11] requires a 3-D tensor, we produce different formats of data for different algorithms.

B. Decoding Algorithms for Comparison

The performance of Tensor FISTA-Net is evaluated on the testing dataset, along with other different decoding algorithms.

Among the algorithms used as benchmarks, GAP-TV [18] adopts the idea of total variance minimization under the GAP, which is a simple model.

SeSCI [20] uses a shearlet transformation to exploit the sparsity hidden in both the frequency and shearlet domains.

λ -net [23] is the state-of-the-art neural-network-based algorithm specifically designed for hyperspectral SCI cameras.

C. Evaluation Metrics

We still calculate the PSNR and SSIM on different scenes of testing data and record the time consumed in the decoding process. We will illustrate the spectral densities of particular

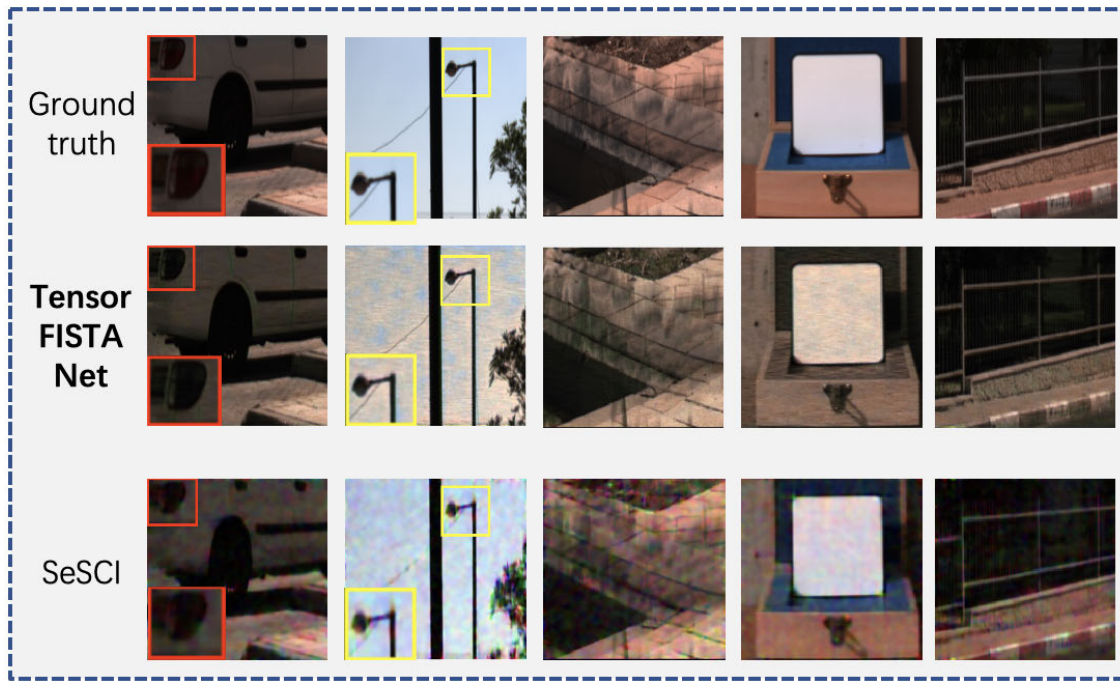


Fig. 11. Selected reconstructed images integrated by RGB channels. From left to right the scenes are car, street lamp, sidewalk, object, and guardrail. From the decoding results, it is clear that Tensor FISTA-Net obtains results with much less noise and blur, especially in the *car* image than SeSCI. Moreover, Tensor FISTA-Net performs better in the consistency of colors than SeSCI, but it suffers from incorrect colors in some specific areas.

TABLE IV
PSNR AND SSIM ON TESTING DATASETS

	<i>Ours</i>	GAP-TV [18]	λ -net [23]	SeSCI [20]
Average	27.87/0.89	24.27/0.74	32.37/0.91	22.45/0.80
Scene1	32.08/0.94	29.48/0.80	36.29/0.93	23.47/0.84
Scene2	20.89/0.85	16.58/0.81	30.07/0.93	21.65/0.81
Scene3	31.41/0.95	21.48/0.77	34.19/0.94	20.64/0.78
Scene4	23.18/0.85	26.49/0.82	28.90/0.90	24.64/0.87
Scene5	31.23/0.91	26.63/0.68	34.58/0.89	24.00/0.80
Scene6	25.20/0.84	22.81/0.61	28.09/0.86	21.26/0.79
Scene7	27.04/0.92	24.95/0.70	36.15/0.94	22.06/0.84
Scene8	26.30/0.90	21.26/0.70	32.64/0.91	21.17/0.73
Scene9	30.53/0.87	29.94/0.81	33.83/0.91	27.25/0.91
Scene10	20.92/0.82	23.04/0.71	28.83/0.88	18.38/0.64

TABLE V
AVERAGE PSNR AND SSIM BEFORE AND AFTER WE ADD PNP METHOD TO OUR *Tensor-FISTA-Net*

Scene	<i>Ours (without PnP)</i>	<i>Ours (with PnP)</i>
Average	27.87/0.89	28.18/0.93
Scene1	32.08/0.94	32.86/0.94
Scene2	20.89/0.85	21.06/0.89
Scene3	31.41/0.95	31.93/0.96
Scene4	23.18/0.85	23.29/0.88
Scene5	31.23/0.91	31.59/0.91
Scene6	25.20/0.84	25.29/0.85
Scene7	27.04/0.92	27.51/0.93
Scene8	26.30/0.90	26.16/0.91
Scene9	30.53/0.87	31.13/0.88
Scene10	20.92/0.82	20.97/0.86

data points on different channels of images and compare them with the original spectral density.

The PSNR and SSIM of testing scenes are shown in Table IV. The table includes ten individual sets of PSNR and SSIM and their average values produced by decoding algorithms. Our proposed Tensor FISTA-Net shows an improvement of 4.43 dB (SeSCI) and 2.61 dB (GAP-TV) in PSNR. Among the results, we can see that λ -net performs

TABLE VI
AVERAGE RECONSTRUCTION TIME (SECONDS)

Algorithm	Average (Seconds)
<i>Tensor FISTA-Net (Ours)</i>	0.30
GAP-TV [18]	22.30
SeSCI [20]	426.45

best in terms of PSNR and SSIM. However, λ -net is restricted to reconstructing images from hyperspectral SCI cameras. Therefore, as a universal method for a real-time decoder, the results of Tensor FISTA-Net are still satisfactory. Table V illustrates the comparison between the decoding results of our Tensor FISTA-Net with and without the plug-and-play method [24] applied. The result observes an increase of 0.3 dB in PSNR and 0.04 in SSIM on average. Therefore, it can be concluded that the plug-and-play method does improve the accuracy of reconstruction results on this hyperspectral image dataset because the dynamical updating mechanism helps control the sparsity penalty more exquisitely.

Table VI shows the decoding time of different algorithms on datasets of hyperspectral SCI cameras. It is clearly illustrated that Tensor FISTA-Net brings a huge improvement in decoding time than other benchmark algorithms. Using Tensor FISTA-Net as the decoding method, a single hyperspectral image with 24 frames in different wavelengths takes 0.3 s to decode, which indicates the decoding speed of Tensor FISTA-Net can reach up to 70 frames per second, which is already sufficient for a real-time decoder.

Table VI clearly shows that SeSCI achieves similar results with Tensor FISTA-Net but with a much longer running time. SeSCI takes about 7 min to reconstruct a single measurement, but Tensor FISTA-Net takes 0.3 s. Our Tensor FISTA-Net applies the DNN to trade training time with decoding time.

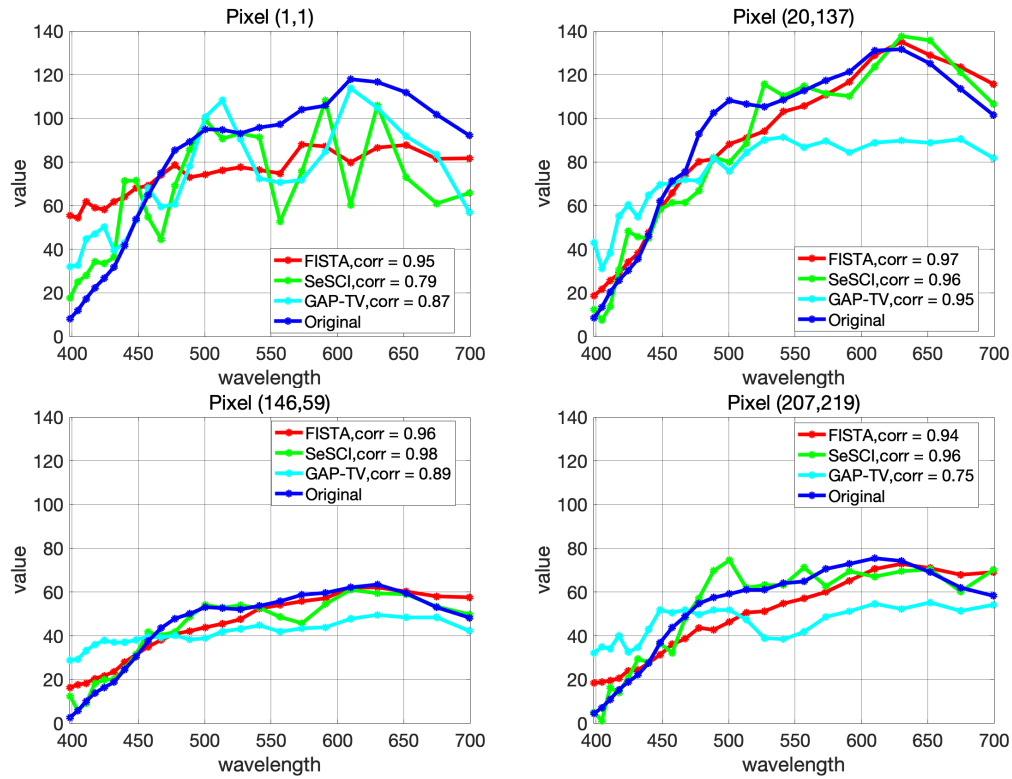


Fig. 12. Spectral densities of four pixels selected in the car image. Tensor FISTA-Net tends to achieve similar values across the range of wavelength, while we can find more fluctuations in the result of SeSCI.

D. Visual Comparisons

In this section, we select several channels of reconstruction results and demonstrate them in a row as a comparison of decoding details, which is shown in Fig. 10. The selected channels are near the average wavelengths of red, green, and blue lights so that they can be integrated into an RGB image via superposition, as shown in Fig. 11. We select five images from different scenarios. In Fig. 11, the five scenarios are (from left to right): car, street lamp, sidewalk, object, and guardrail. According to the evaluation metrics and visual reconstruction results, we can conclude that: Compared with *SeSCI*, Tensor FISTA-Net reduces a lot of noise and blurry places in the testing images, achieving better PSNR and SSIM results. However, Tensor FISTA-Net tends to reconstruct similar densities through the range of wavelengths instead of restoring its original tone. Therefore, the decoding results seem to have a red shift in general.

E. Spectral Density Comparison

In the last subsection of the evaluation, we dig into the spectral density of single pixel points and calculate the correlation of the density curves throughout the range of wavelengths. The spectral density comparison method is a useful method to evaluate the coherence of decoding results throughout the range of wavelengths. We randomly select four pixels from the image: (1, 1), (20, 137), (146, 59), (207, 219). The spectral densities of these pixels obtained from different decoding algorithms are shown in Fig. 12.

From the spectral density figure of those points, we can conclude that the decoding results of *SeSCI* [20] tend to be

highly volatile, while those of Tensor FISTA-Net tend to be more stable throughout the channels. Combining the output images with the correlation, we can see that Tensor FISTA-Net provides better results than [20].

The spectral density curves dig into the nature of the different reconstruction algorithms: [20] uses shearlet transformation which mainly extracts the contour information as low-frequency signals, while its high-frequency signals mainly show the texture information of the images. Therefore, in the spectral density reconstructed by Yang et al. [20], the reconstructed low-frequency signals are more accurate.

In Tensor FISTA-Net, the decoding results seem to have a much smaller vibration than the original spectral density. This may result from the nature of convolutional neural networks which tends to learn the sparsest solution in the transformation domain.

VII. CONCLUSION

In this article, we have proposed a Tensor FISTA-Net as a real-time decoder for SCI cameras, by unfolding the tensor FISTA algorithm into a DNN. We put forward a server-based training model to save training time. To meet the requirement of real-time in the inference stage, we convert the calculations from matrix forms into tensor forms and adopt several optimizing techniques. The experimental results on both the video and hyperspectral SCI datasets show that Tensor FISTA-Net provides better decoding quality than the state-of-the-art algorithms and the decoding speed exceeds the threshold of real-time decoding. In addition, Tensor FISTA-Net consumes much less memory, and therefore it is suitable for real-time applications on IoT devices.

Since the training of neural networks can be performed by a cloud server, in the future it would be interesting to explore the homomorphic property [46] for privacy-sensitive applications, for example, a federated learning application [47].

REFERENCES

- [1] D. L. Donoho, "Compressed sensing," *IEEE Trans. Inf. Theory*, vol. 52, no. 4, pp. 1289–1306, Apr. 2016.
- [2] E. J. Candès, J. Romberg, and T. Tao, "Robust uncertainty principles: Exact signal reconstruction from highly incomplete frequency information," *IEEE Trans. Inf. Theory*, vol. 52, no. 2, pp. 489–509, Feb. 2006.
- [3] E. J. Candès and T. Tao, "Near-optimal signal recovery from random projections: Universal encoding strategies?" *IEEE Trans. Inf. Theory*, vol. 52, no. 12, pp. 5406–5425, Dec. 2006.
- [4] C. Kwan et al., "Deep learning-based target tracking and classification for low quality videos using coded aperture cameras," *Sensors*, vol. 19, no. 17, p. 3702, Aug. 2019.
- [5] C. Kwan, D. Gribben, A. Rangamani, T. Tran, J. Zhang, and R. Etienne-Cummings, "Detection and confirmation of multiple human targets using pixel-wise code aperture measurements," *J. Imag.*, vol. 6, no. 6, p. 40, May 2020.
- [6] P. Llull et al., "Coded aperture compressive temporal imaging," *Opt. Exp.*, vol. 21, no. 9, pp. 10526–10545, May 2013.
- [7] M. E. Gehm, R. John, D. J. Brady, R. M. Willett, and T. J. Schulz, "Single-shot compressive spectral imaging with a dual-disperser architecture," *Opt. Exp.*, vol. 15, no. 21, pp. 14013–14027, 2007.
- [8] A. Wagadarikar, R. John, R. Willett, and D. Brady, "Single disperser design for coded aperture snapshot spectral imaging," *Appl. Opt.*, vol. 47, no. 10, pp. B44–B51, 2008.
- [9] J. Yang et al., "Video compressive sensing using Gaussian mixture models," *IEEE Trans. Image Process.*, vol. 23, no. 11, pp. 4863–4878, Nov. 2014.
- [10] J. Yang et al., "Compressive sensing by learning a Gaussian mixture model from measurements," *IEEE Trans. Image Process.*, vol. 24, no. 1, pp. 106–119, Jan. 2015.
- [11] Y. Liu, X. Yuan, J. Suo, D. J. Brady, and Q. Dai, "Rank minimization for snapshot compressive imaging," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 12, pp. 2990–3006, Dec. 2019.
- [12] A. Beck and M. Teboulle, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM J. Imag. Sci.*, vol. 2, no. 1, pp. 183–202, 2009.
- [13] J. Zhang and B. Ghanem, "ISTA-Net: Interpretable optimization-inspired deep network for image compressive sensing," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1828–1837.
- [14] F. Jiang, X.-Y. Liu, H. Lu, and R. Shen, "Efficient multi-dimensional tensor sparse coding using t-linear combination," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 1–8.
- [15] J. Ma, X.-Y. Liu, Z. Shou, and X. Yuan, "Deep tensor ADMM-Net for snapshot compressive imaging," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 10223–10232.
- [16] Y. Zhang, X.-Y. Liu, B. Wu, and A. Walid, "Video synthesis via transform-based tensor neural network," in *Proc. 28th ACM Int. Conf. Multimedia*, Oct. 2020, pp. 2454–2462.
- [17] X. Han, B. Wu, Z. Shou, X. Y. Liu, and L. Kong, "Tensor FISTA-Net for real-time snapshot compressive imaging," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 7, pp. 10933–10940.
- [18] X. Yuan, "Generalized alternating projection based total variation minimization for compressive sensing," in *Proc. IEEE Int. Conf. Image Process.*, Sep. 2016, pp. 2539–2543.
- [19] L. Yang and X.-Y. Liu, "Tensor nuclear-norm minimization for snapshot compressive imaging cameras," in *Proc. IEEE MIT Undergraduate Res. Technol. Conf. (URTC)*, Oct. 2019, pp. 1–4.
- [20] P. Yang, L. Kong, X.-Y. Liu, X. Yuan, and G. Chen, "Shearlet enhanced snapshot compressive imaging," *IEEE Trans. Image Process.*, vol. 29, pp. 6466–6481, 2020.
- [21] Z. Zhang, Y. Liu, J. Liu, F. Wen, and C. Zhu, "AMP-Net: Denoising-based deep unfolding for compressive image sensing," *IEEE Trans. Image Process.*, vol. 30, pp. 1487–1500, 2021.
- [22] M. Iliadis, L. Spinoulas, and A. K. Katsaggelos, "Deep fully-connected networks for video compressive sensing," *Digit. Signal Process.*, vol. 72, pp. 9–18, Jan. 2018.
- [23] X. Miao, X. Yuan, Y. Pu, and V. Athitsos, "Lambda-Net: Reconstruct hyperspectral images from a snapshot measurement," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 4058–4068.
- [24] X. Yuan, Y. Liu, J. Suo, and Q. Dai, "Plug-and-play algorithms for large-scale snapshot compressive imaging," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1447–1457.
- [25] H. Qiu, Y. Wang, and D. Meng, "Effective snapshot compressive-spectral imaging via deep denoising and total variation priors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 9127–9136.
- [26] Z. Zha, B. Wen, X. Yuan, J. Zhou, C. Zhu, and A. C. Kot, "A hybrid structural sparsification error model for image restoration," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 4451–4465, Sep. 2022.
- [27] Z. Zha, B. Wen, X. Yuan, J. Zhou, C. Zhu, and A. C. Kot, "Low-rankness guided group sparse representation for image restoration," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Feb. 7, 2022, doi: 10.1109/TNNLS.2022.3144630.
- [28] Z. Cheng et al., "BIRNAT: Bidirectional recurrent neural networks with adversarial training for video snapshot compressive imaging," in *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXIV*. Springer, 2020, pp. 258–275.
- [29] Z. Cheng et al., "Recurrent neural networks for snapshot compressive imaging," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 2264–2281, Feb. 2023.
- [30] Y. Hitomi, J. Gu, M. Gupta, T. Mitsunaga, and S. K. Nayar, "Video from a single coded exposure photograph using a learned over-complete dictionary," in *Proc. Int. Conf. Comput. Vis.*, Nov. 2011, pp. 287–294.
- [31] X. Yuan et al., "Low-cost compressive sensing for color video and depth," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 3318–3325.
- [32] W. S. Boyle and G. E. Smith, "Charge coupled semiconductor devices," *Bell Syst. Tech. J.*, vol. 49, no. 4, pp. 587–593, Apr. 1970.
- [33] M. F. Tompsett, G. F. Amelio, and G. E. Smith, "Charge coupled 8-bit shift register," *Appl. Phys. Lett.*, vol. 17, no. 3, pp. 111–115, Aug. 1970.
- [34] R. Tibshirani, "Regression shrinkage and selection via the lasso," *J. Roy. Statist. Soc. B, Methodol.*, vol. 58, no. 1, pp. 267–288, 1996.
- [35] S. Jalali and X. Yuan, "Compressive imaging via one-shot measurements," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2018, pp. 416–420.
- [36] B. Wu, T. Mei, W.-H. Cheng, and Y. Zhang, "Unfolding temporal dynamics: Predicting social media popularity using multi-scale temporal decomposition," in *Proc. 13th AAAI Conf. Artif. Intell.*, 2016, pp. 1–7.
- [37] B. Wu, W.-H. Cheng, Y. Zhang, and T. Mei, "Time matters: Multi-scale temporalization of social media popularity," in *Proc. 24th ACM Int. Conf. Multimedia*, Oct. 2016, pp. 1336–1344.
- [38] C. Dong, C. C. Loy, K. He, and X. Tang, "Learning a deep convolutional network for image super-resolution," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 184–199.
- [39] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.
- [40] D. L. Donoho, "De-noising by soft-thresholding," *IEEE Trans. Inf. Theory*, vol. 41, no. 3, pp. 613–627, May 1995.
- [41] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.
- [42] X. Yuan, D. J. Brady, and A. K. Katsaggelos, "Snapshot compressive imaging: Theory, algorithms, and applications," *IEEE Signal Process. Mag.*, vol. 38, no. 2, pp. 65–88, Mar. 2021.
- [43] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.
- [44] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [45] B. Arad and O. Ben-Shahar, "Sparse recovery of hyperspectral signal from natural RGB images," in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part VII 14*. Springer, 2016, pp. 19–34.
- [46] X.-Y. Liu, Z. Li, and X. Wang, "Homomorphic matrix completion," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 12197–12210.
- [47] L. Kong, X.-Y. Liu, H. Sheng, P. Zeng, and G. Chen, "Federated tensor mining for secure industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 2144–2153, Mar. 2020.



Xiao-Yang Liu (Member, IEEE) received the B.Eng. degree in computer Science from the Huazhong University of Science and Technology, Wuhan, China, in 2010, and the M.S. degree in electrical engineering from Columbia University, New York, NY, USA, in 2018, where he is currently pursuing the Ph.D. degree with the Department of Electrical Engineering. He graduated from the Ph.D. program with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China, 2017.

His research interests include tensor theory, high-performance tensor computation, deep learning, optimization algorithms, big data analysis, and data privacy.



Linghe Kong (Senior Member, IEEE) received the B.Eng. degree in automation from Xidian University, Xi'an, China, in 2005, the master's degree in telecommunication from Telecom SudParis, France, in 2007, and the Ph.D. degree in computer science from Shanghai Jiao Tong University, Shanghai, China, in 2013.

He was a Post-Doctoral Researcher with Columbia University, New York, NY, USA, McGill University, Montréal, QC, Canada, and the Singapore University of Technology and Design, Singapore. He is currently a Professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include the Internet of Things, 5G, blockchain, and mobile computing.



Qifan Huang received the B.S. degree in mathematics and physics from Tsinghua University, Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with the Institute of Software, Chinese Academy of Sciences and the University of Chinese Academy of Sciences, Beijing.

His research interests include tensor theory, quantum software, and quantum electronic design and automation.



Anwar Walid (Fellow, IEEE) received the B.S. degree in electrical engineering from New York University, New York, NY, USA, in 1986, and the Ph.D. degree in electrical engineering from Columbia University, New York, in 1991.

He was the Head of the Mathematics of Systems Research Department, Nokia Bell Labs, Murray Hill, NJ, USA. He is currently an Adjunct Professor with the Electrical Engineering Department, Columbia University.

Dr. Walid is an Elected Member of Tau Beta Pi National Engineering Honor Society and IFIP Working Group 7.3. He received the 2017 IEEE Communication Society William R. Bennett Prize and the 2019 ACM SIGCOMM Networking Systems Award. He served as an Associate Editor for the IEEE/ACM TRANSACTIONS ON NETWORKING (TON), the IEEE TRANSACTIONS ON CLOUD COMPUTING, and *IEEE Network Magazine*.



Xiaochen Han received the B.S.E. degree in computer science from Shanghai Jiao Tong University, Shanghai, China, in 2020, and the M.S. degree in computer vision from Carnegie Mellon University, Pittsburgh, PA, USA, in 2022.

He is currently a Research Engineer working with the Meta Reality Labs, Burlingame, CA, USA. His research interests include deep learning, computer vision, and machine learning.



Xiaodong Wang (Fellow, IEEE) received the Ph.D. degree in electrical engineering from Princeton University, Princeton, NJ, USA, in 1998.

He is currently a Professor of electrical engineering with Columbia University, New York, NY, USA. He has authored *Wireless Communication Systems: Advanced Techniques for Signal Reception* (Prentice Hall, 2003). His current research interests include wireless communications, statistical signal processing, and genomic signal processing. His research interests fall in the general areas of computing, signal processing, and communications. He has authored extensively in these areas.

Dr. Wang has served as an Associate Editor for the IEEE TRANSACTIONS ON COMMUNICATIONS, the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, the IEEE TRANSACTIONS ON SIGNAL PROCESSING, and the IEEE TRANSACTIONS ON INFORMATION THEORY. He is an ISI Highly Cited Author. He received the 1999 NSF CAREER Award, the 2001 IEEE Communications Society and Information Theory Society Joint Paper Award, and the 2011 IEEE Communication Society Award for Outstanding Paper on New Communication Topics.



Bo Wu received the Ph.D. degree in computer science from the Chinese Academy of Sciences (ICT, CAS), Beijing, China, in 2017.

He is currently a Researcher with MIT-IBM Watson AI Lab, MA, USA. He was a Research Scientist with Columbia University. His current research interests are deep learning, multimodal learning, computer vision, and natural language learning.

Dr. Wu received the IBM Level-A Accomplishment Award, the ACL 2020 Best Demo Paper Award, the ICIP Prediction Challenge 2020 (Top1), and NIST TAC SM-KBP 2019 (Top1).

and NIST TAC SM-KBP 2019 (Top1).